Sutra Mini: a reliable accounting push

Shree Radha Studio · Full Stack and Applied-AI Engineer · Case study (Round 2)

Time: 3 to 4 hours. Please do not spend more; we score correctness, not polish. **Submit within three days** of receiving this brief.
Stack: Go for the backend, Neon (Postgres, free tier) for the database, Next.js for the one page. Use any coding agents or AI tools you like; we want to see how you work with them.
Deliver: a public GitHub repo, a README that runs from a clean machine, `AGENT_NOTES.md`, and a 3-minute screen recording. Reply to the invite email with the links.

The problem

We make women's ethnicwear. Fabric goes out to embroidery and stitching job-workers and comes back in parts over days. Every receipt has an accounting consequence that must reach our accounting software, whose API is slow and fails without warning. Sutra is the layer that makes this reliable. Build the smallest honest version of it.

What we give you

Create these tables on Neon (you may change the design, but then write migrations). Seed 5 SKUs with image URLs, 2 job-workers and 4 open issues.

```
skus(id, code, name, image_url)
parties(id, name, phone)                          -- job-workers
jobwork_issues(id, party_id, sku_id, qty, issued_on)
jobwork_receipts(id, issue_id, qty, received_on, idempotency_key)
vouchers(id, type, source_id, amount_paise, status,   -- status: pending | pushed | failed
         attempts, last_error, client_ref, external_ref, created_at, updated_at)
```

Build (Go on Neon)

1. `POST /jobwork/receipts` with `{issue_id, qty, received_on}`. Rules: a receipt may never make total received exceed the issued quantity, **even when several requests arrive at the same moment**; the voucher row is created **in the same transaction** as the receipt; the request accepts an `Idempotency-Key` header, and replaying the same key returns the same result and creates nothing twice. Enforce that in the database, not in memory.
2. `GET /issues/{id}`: issued, received, outstanding. `GET /vouchers?status=`. `POST /vouchers/{id}/retry`.
3. A tiny **mock accounting server** with one endpoint, `POST /api/vouchers`, that misbehaves on purpose: 20% of calls return HTTP 500; 10% hang for 5 seconds and then succeed; a duplicate `client_ref` returns HTTP 409 with the original `external_ref`.
4. A **sync worker** that pushes pending vouchers to the mock: retries with backoff; a voucher that keeps failing ends `failed` with a reason; a voucher is **never posted twice**, even if the worker crashes between the HTTP call and the database update; two instances of the worker can run at once safely; the HTTP client has timeouts and the 5-second hang does not block other vouchers.

Build (Next.js, one page)

`/sync`: vouchers grouped by status with attempt count and failure reason, and a Retry button. Keep it plain; we have designs for the real product.

Tests

At least two: one proving overshoot is impossible under concurrent receipts, one proving an idempotency-key replay creates nothing. They must fail if the guarantee is broken.

Stretch, only if you finished early (small bonus)

`POST /ask`: answer "how much of SKU X is still with job-worker Y?" with an LLM using read-only tools over the database, each tool call logged. Stub the model if you have no key and say so.

How we evaluate

Area	What matters
Transactional correctness	Receipt and voucher in one transaction; no overshoot under concurrency; idempotency enforced by Postgres
Reliability of the push	Crash-safe, two instances safe, backoff, duplicates reconciled through the 409 path, timeouts
Go and schema quality	Small handlers, context propagation, errors surfaced, money stored safely, migrations
Tests	The two tests exist, run, and mean something
Notes and recording	Candid AGENT_NOTES.md: tools used, what they got wrong, one thing you caught. Recording shows a retry succeeding and a duplicate rejected

We will run your README on a clean machine, replay a request with the same key, fire ten concurrent receipts, kill the worker during the 5-second hang and restart it, then run two workers at once. Design for that.

Rules

- Your own work, with any tools you like. In the technical round you will change one behaviour live, so understand every line you submit.
- No secrets committed: keys in `.env`, ship a `.env.example`.
- Questions are welcome; reply to the invite email. A good question counts in your favour.

Shree Radha Studio · Gandhi Nagar, Delhi · On-site role · This is the kind of work you would do from week one.